

apache2

Saïd El Mamouni

Un **serveur HTTP** permet à un serveur web de communiquer avec un navigateur.

Il utilise le protocole **HTTP(S)** et ses extensions (**WebDAV, etc.**).

Apache est probablement le serveur HTTP le plus populaire.

Autres serveurs : Nginx,

Il est produit par la Apache Software Foundation.

C'est un logiciel libre fourni sous la licence spécifique Apache.

On utilise généralement Apache en conjonction avec d'autres logiciels, permettant d'interpréter du code et d'accéder à des bases de données. Le cas le plus courant est celui d'un serveur LAMP (Linux Apache MySQL PHP).

L'Hypertext Transfer Protocol (« protocole de transfert hypertexte »)

HTTP est un protocole de communication client-serveur développé pour le WWW.

HTTPS (secured) est la variante du HTTP sécurisée par l'usage des protocoles SSL ou TLS.

HTTP est un protocole de la couche application.

HTTP utilise le protocole TCP comme couche de transport.

Un serveur HTTP utilise par défaut le port 80 (443 pour HTTPS).

Les clients HTTP les plus connus sont les navigateurs Web permettant à un utilisateur d'accéder à un serveur contenant les données. Il existe aussi des systèmes pour récupérer automatiquement le contenu d'un site tel que les aspirateurs de site Web ou les robots d'indexation.

Installation

Pour installer Apache seul

```
$sudo apt install apache2
```

Pour installer Apache avec PHP et MySQL

```
$sudo apt install apache2 php libapache2-mod-php mysql-server php-mysql
```

Pour installer Apache avec PHP et Mariadb

```
$sudo apt install apache2 php libapache2-mod-php mariadb-server php-mysql
```

(attention le paquet php installe directement la dernière version de php)

```
$sudo apt install apache2 php libapache2-mod-php mysql-server php-mysql
```

MySQL

MySQL est un **système de gestion de bases de données relationnelles** (SGBDR)

MySQL est distribué sous une double licence GPL et propriétaire (Oracle)

-

Depuis mai 2009, son créateur Michael Widenius a créé **MariaDB** (Maria est le prénom de sa deuxième fille) pour continuer son développement en tant que projet Open Source.

```
$sudo apt install apache2 php libapache2-mod-php mysql-server php-mysql
```

Ce paquet fournit un module MySQL pour PHP

```
$sudo apt install apache2 php libapache2-mod-php mysql-server php-mysql
```

libapache2-mod-php - Ce paquet fournit le module PHP pour le serveur web Apache 2
langage de script inclus dans du HTML et exécuté côté serveur

PHP (préprocesseur hypertexte) est un langage de script libre très utilisé et généraliste. Il est particulièrement adapté au développement web et peut être embarqué dans du HTML.

Lancement

Apache 2 se lance par défaut **dès son installation**
Apache 2 se relance automatiquement **à chaque démarrage.**

Pour empêcher cela :

```
$sudo systemctl disable apache2
```

Pour ré-automatiser cela :

```
$sudo systemctl enable apache2
```

Lancement

Apache 2 se lance par défaut **dès son installation**
Apache 2 se relance automatiquement **à chaque démarrage.**

Pour empêcher cela :

```
$sudo systemctl disable apache2
```

Votre serveur doit fonctionner et être accessible à l'adresse **http://localhost** (à partir de la même machine).

localhost se réfère à l'ordinateur local.

Localhost est associé à l'adresse IP **127.0.0.1** en IPV4 et **::1** en IPv6

Cette notion est utilisée pour simuler des communications IP sur un ordinateur mais sans envoyer de messages sur le réseau.

L'interface réseau virtuelle utilisée dans cette situation se nomme l'interface de **loopback** (abrégée par **lo** sous Unix) ou **boucle locale**

Autres commandes utiles

Pour arrêter apache2 : **sudo systemctl stop apache2**

Pour lancer apache2 : **sudo systemctl start apache2**

Pour relancer apache2 : **sudo systemctl restart apache2**

Pour recharger la configuration d'apache2 : **sudo systemctl reload apache2**

Pour voir la version d'Apache utilisée : **sudo apache2ctl -v**

Pour tester l'ensemble de la configuration d'Apache : **sudo apache2ctl -t**

Pour tester la configuration des hôtes virtuels : **sudo apache2ctl -t -D DUMP_VHOSTS**

Pour voir les modules d'Apache chargés : **sudo apache2ctl -M**

Mode de fonctionnement sommaire

Au démarrage :

- Apache charge les fichiers de configuration
- Se met en écoute (listen) sur certains ports de ses interfaces réseaux.

Localement lors d'une recherche depuis un navigateur, une requête est effectuée.

- Le navigateur résout le nom de domaine (il obtient l'adresse IP du serveur).
- envoie une requête HTTP avec la méthode GET à l'IP du serveur sur le port 80 (ou HTTPS sur le port 443)

Le serveur apache reçoit la requête et la traite :

- il va chercher un contenu dans un fichier ou lance un script qui va générer un contenu.
- Renvoie ce contenu à l'IP du navigateur depuis le même port (80 ou 443).
- Le navigateur traite le contenu et le rend accessible à l'internaute (en l'affichant à l'écran par ex.).

Fichiers de configuration

Apache peut instancier plusieurs sites ou services internet, son répertoire de configuration est **/etc/apache2**.

sites-available contient les fichiers de configuration des sites disponibles

sites-enabled contient des liens symboliques vers les configurations, dans site-available, des sites activés

conf-available contient les fichiers de configuration des autres services disponibles

conf-enabled contient des liens symboliques vers les configurations, dans conf-available, des autres services activés

mods-available contient les fichiers de configuration des modules d'Apache disponibles

mods-enabled contient des liens symboliques vers les configurations, dans mods-available, des modules activés

Toute la configuration se fait dans les sous dossiers xxx-available

Activation/désactivation des configurations

a2en pour Apache 2 enable - a2dis pour Apache 2 disable

Commandes de création ou suppression de liens symbolique dans les répertoires xxx-enabled

- sudo a2ensite [configuration d'un site à activer]
- sudo a2dissite [configuration d'un site à désactiver]
- sudo a2enconf [configuration d'un service à activer]
- sudo a2disconf [configuration d'un service à désactiver]
- sudo a2enmod [configuration d'un module à activer]
- sudo a2dismod [configuration d'un module à désactiver]

=> sudo systemctl reload apache2 (pour charger les modifications)

Configuration des ports d'écoute

Le fichier `/etc/apache2/ports.conf` permet de spécifier les ports à écouter.

→ Par défaut, apache écoute sur le port 80 en HTTP et 443 pour HTTPS (si le module SSL est activé).

Hôtes virtuels

Avec Apache, chaque site ou application web correspond en principe à un hôte virtuel (VirtualHost en anglais).

Chaque hôte virtuel est défini par un fichier de configuration indépendant, qu'on trouve ou qu'on crée dans le répertoire `/etc/apache2/sites-available/`, 2 hôtes virtuels sont installés par défaut.

000-default.conf, le premier VirtualHost

000-default.conf

```
<VirtualHost *:80>  
    ServerAdmin webmaster@localhost  
    DocumentRoot /var/www/html  
    ErrorLog ${APACHE_LOG_DIR}/error.log  
    CustomLog ${APACHE_LOG_DIR}/access.log combined  
</VirtualHost>
```

directive	description
<VirtualHost *:80>	On déclare l'hôte virtuel. Il doit répondre aux requêtes qui s'adressent à toutes les adresses IP (*) du serveur sur le port 80. On peut ici éventuellement spécifier une adresse IP unique à laquelle répondra Apache pour un hôte virtuel donné (dans le cas où le serveur possède plusieurs adresses IP), ou choisir de répondre au port 443 (pour HTTPS) ou à un port exotique (à condition de l'écouter).
ServerAdmin webmaster@localhost	C'est le courriel de l'administrateur système. Cette directive n'est ni très utile, ni indispensable.
DocumentRoot /var/www/html	C'est le chemin absolu vers l'emplacement local (sur l'espace disque du serveur) qui sera la racine de l'hôte. Apache recherche en premier un fichier index.html ou index.php à afficher par défaut à l'emplacement spécifié. On constate que c'est donc bien cet hôte virtuel qui affiche le message It Works! contenu dans le fichier /var/www/html/index.html.
ErrorLog \${APACHE_LOG_DIR}/error.log CustomLog \${APACHE_LOG_DIR}/access.log combined	Ce sont des directives relatives au log d'erreur et au log d'accès de cet hôte virtuel. Ceux-ci seront donc écrits dans le répertoire APACHE_LOG_DIR, qui est par défaut /var/log/apache2 sur ubuntu.
</VirtualHost>	Fin de la section concernant cet hôte virtuel.

default-ssl.conf, Le second VirtualHost.

- Il contient une configuration par défaut pour HTTPS.
- Il écoute donc sur le port 443 et énonce des directives spécifiques à l'usage de SSL.

La racine de l'hôte (directive DocumentRoot) correspond également à /var/www/html.

Même si, dans le cas d'un seul site, il peut être directement déployé dans /var/www/html, il est conseillé de créer un nouvel hôte virtuel.

Exemples de configurations :

<https://httpd.apache.org/docs/trunk/fr/vhosts/examples.html>

directive	description
<code><VirtualHost *:80></code>	On accepte les connexions sur n'importe quelle IP du serveur (*) sur le port 80.
<code>ServerName example.com</code>	Cet hôte virtuel sera seulement appelé pour le nom de domaine example.com...
<code>ServerAlias www.example.com</code>	...ainsi que pour le sous-domaine www.example.com. On peut spécifier ici d'autres noms de domaine en les séparant par un espace. On peut aussi utiliser *.example.com pour inclure tous les sous-domaines.
<code>DocumentRoot "/var/www/example"</code>	On placera les fichiers du site dans le répertoire /var/www/example
<code><Directory "/var/www/example"></code>	On spécifie dans cette section des règles pour le répertoire /var/www/example sous cet hôte virtuel.

directive	description
<Directory "/var/www/example">	On spécifie dans cette section des règles pour le répertoire /var/www/example sous cet hôte virtuel.
Options +FollowSymLinks	Apache suivra les liens symboliques qu'il trouvera dans ce répertoire (et
AllowOverride all	On pourra inclure une configuration personnalisée via un fichier .htaccess.
Require all granted	Tous les visiteurs pourront accéder au contenu de ce répertoire. Voir la documentation officielle pour modifier ce comportement. Pour des raisons de sécurité ou de privacité on peut par exemple limiter l'accès au serveur à seulement une ou certaines adresses IP avec une directive du type Require ip 192.168.1.10.
ErrorLog /var/log/apache2/error.example.com.log CustomLog /var/log/apache2/access.example.com.log combined	Il est pratique d'avoir des logs séparés pour chaque hôte virtuel, afin de ne pas mélanger toutes les informations.

Activation de la configuration

→ `sudo a2ensite [nom du fichier sans son extension]`.

Par exemple :

→ `sudo a2ensite example.com`

On recharge ensuite la configuration d'Apache :

→ `sudo systemctl reload apache2`

==== Sécurisation ====

Commande pour récupérer le nom de sa machine:

```
$ hostname
```

Installer le paquet d'Openssl:

```
$ sudo apt install openssl
```

Création d'un certificat:

```
$ openssl req -new -x509 -days 365 -nodes -out /etc/ssl/certs/mailserver.crt -keyout  
/etc/ssl/private/mediawiki.key
```

Répondre à cette question par:

```
Common Name (eg, YOUR name) [] : <hostname>.sio.greta.pornic.fr
```

Puis activer le ssl pour apache:

```
a2enmod ssl
```

Créer un fichier dans /etc/apache2/sites-available/mediawiki.conf:

```
<VirtualHost *:80>
```

```
    ServerName toto.sio.greta.pornic.fr
```

```
    ServerAlias toto
```

```
    ServerAdmin titi@ecomail.fr
```

```
    DocumentRoot /var/www/html/mediawiki
```

```
        <Directory /var/www/html/mediawiki>
```

```
            Options -Indexes +FollowSymLinks +MultiViews
```

```
            AllowOverride none
```

```
            Require all granted
```

```
        </Directory>
```

```
    ErrorLog /var/log/apache2/error.mediawiki.com.log
```

```
    CustomLog /var/log/apache2/access.mediawiki.com.log combined
```

```
</VirtualHost>
```

```
<VirtualHost *:443>
```

```
DocumentRoot /var/www/html/mediawiki
```

```
ServerName toto.sio.greta.pornic.fr
```

```
ServerSignature Off
```

```
ErrorLog ${APACHE_LOG_DIR}/error_mediawiki.log
```

```
LogLevel info
```

```
CustomLog ${APACHE_LOG_DIR}/access_mediawiki.log combined
```

```
SSLEngine on
```

```
SSLCertificateFile /etc/ssl/certs/mailserver.crt
```

```
SSLCertificateKeyFile /etc/ssl/private/mailserver.key
```

```
</VirtualHost>
```

On active le site avec la commande:

```
a2ensite mediawiki
```

Puis on redémarre apache2 avec la commande:

```
systemctl reload apache2
```

Configuration Apache

En pratique avec Apache (sous Debian 8), voici une configuration SSL avancée :

```
SSLProtocol All -SSLv2 -SSLv3
```

```
SSLCipherSuite AES128+EECDH:AES128+EDH
```

```
SSLHonorCipherOrder On
```

```
SSLCompression off
```

```
#SSLSessionTickets off
```

```
SSLSessionCache shmcb:/var/log/apache2/ssl_sessioncache(10240000)
```

```
SSLUseStapling yes
```

```
SSLStaplingReturnResponderErrors off
```

```
SSLStaplingCache shmcb:/var/log/apache2/ssl_staplingcache(2048000)
```

```
Header always set Strict-Transport-Security "max-age=15552000"
```

Configuration Apache

Voici une configuration minimale :

SSLEngine on

SSLCertificateFile /etc/ssl/certs/secure.crt

SSLCertificateKeyFile /etc/ssl/private/secure.key

#SSLCertificateChainFile /etc/ssl/certs/certificates_chain.CA.crt

SSLProtocol All -SSLv2 -SSLv3

SSL/TLS

SSL (Secure Sockets Layer) est un protocole de sécurisation des échanges réseau.

Il est basé sur le principe de clé privée et clé publique signée par une autorité de certification.

On parle désormais de SSL/TLS car les normes récentes ont renommé SSL en TLS (Transport Layer Security)

SSL est un protocole qui s'intercale entre TCP/IP et les applications qui s'appuient sur TCP.

Une session SSL se déroule en deux temps :

→ 1. une phase de poignée de mains (handshake) durant laquelle le client et le serveur s'identifient, conviennent du système de chiffrement et d'une clé qu'ils utiliseront par la suite.

→ 2. la phase de communication proprement dite durant laquelle les données échangées sont compressées, chiffrées et signées.

L'identification durant la poignée de mains est assurée à l'aide de certificats X509.

- OpenSSL est une boîte à outils cryptographiques implémentant les protocoles SSL et TLS qui offre :
 - 1. une bibliothèque de programmation en C permettant de réaliser des applications client/serveur sécurisées s'appuyant sur SSL/TLS.
 - 2. une commande en ligne (OpenSSL) permettant :
 - la création de clés RSA, DSA (signature)
 - la création de certificats X509
 - le calcul d'empreintes (MD5, SHA, RIPEMD160, ...)
 - le chiffrement et déchiffrement (RSA, DES, IDEA, RC2, RC4, Blowfish, ...)
 - la réalisation de tests de clients et serveurs SSL/TLS
 - la signature et le chiffrement de courriers (S/MIME)

Pour connaître toutes les fonctionnalités de openssl : `man openssl`.

SSL et TLS : rappel historique

SSL et TLS sont deux protocoles cryptographiques qui permettent l'authentification, et le chiffrement des données qui transitent entre des serveurs, des machines et des applications en réseau (notamment lorsqu'un client se connecte à un serveur Web).

Le SSL est le prédécesseur du TLS. Au fil du temps, de nouvelles versions de ces protocoles ont vu le jour pour faire face aux vulnérabilités et prendre en charge des suites et des algorithmes de chiffrement toujours plus forts, toujours plus sécurisés.

Initialement développé par Netscape, le SSL sort en 1995 dans sa version SSL 2.0 (le SSL 1.0 n'étant jamais sorti). Mais après la découverte de plusieurs vulnérabilités en 1996, la version 2.0 est vite remplacée par le SSL 3.0. Remarque : les versions 2.0 et 3.0 sont parfois libellées ainsi : SSLv2 et SSLv3.

Basé sur le SSL 3.0, le TLS est introduit en 1999 comme la nouvelle version du SSL.

Actuellement, la version du TLS utilisée est la v.1.2 – Le TLS v.1.3 n'étant pas finalisé.

Quel protocole utiliser : SSL ou TLS ?

Les versions SSL 2.0 et 3.0 ont toutes deux été désapprouvées par l'IETF (en 2011 et 2015, respectivement).

Des vulnérabilités (comme POODLE, et DROWN) ont été et continuent d'être découvertes dans les protocoles SSL désapprouvés. La plupart des navigateurs récents dégradent l'expérience utilisateur (cadenas ou préfixe HTTPS barré dans la barre d'URL, affichage d'avertissements de sécurité) lorsqu'ils rencontrent un serveur Web qui utilise d'anciens protocoles. **Nous vous recommandons donc de désactiver les versions SSL 2.0 et 3.0 dans votre configuration serveur pour ne conserver que les protocoles TLS.**

Avant de songer à remplacer vos certificats SSL existants par des certificats TLS, rappelons que les certificats ne sont pas dépendants des protocoles. En clair, vous n'avez pas à utiliser un certificat TLS plutôt qu'un certificat SSL. Si de nombreux fournisseurs ont tendance à parler de « Certificat SSL/TLS », il serait sans doute plus exact de parler de « certificats à utiliser avec SSL et TLS », puisque les protocoles sont déterminés par votre configuration serveur, pas par les certificats en tant que tels.

L'expression « Certificats SSL », à ce jour la plus répandue, devrait cependant perdurer, même si le terme TLS commence à percer. L'expression « SSL/TLS » est un compromis fréquent, jusqu'à ce que l'emploi de TLS se soit généralisé.

Alors quelle différence entre les protocoles SSL et TLS ? Dans le langage courant, les différences sont minimales, et le terme SSL reste largement utilisé. Mais dans votre configuration serveur, les différences se traduisent au niveau des vulnérabilités, des suites de chiffrement obsolètes et des avertissements de sécurité du navigateur. Sur vos serveurs, seuls les protocoles TLS doivent être activés.

<https://fr.wikipedia.org/wiki/Hosts>
<https://doc.ubuntu-fr.org/hosts>
https://doc.ubuntu-fr.org/tutoriel/lamp_repertoires_de_travail
https://doc.ubuntu-fr.org/apache2#creation_d_hotes_virtuels
<https://wpformation.com/wordpress-http-https/>
<https://wiki.evolix.org/HowtoSSL>
<http://www.finalclap.com/faq/414-certificat-ssl-https>
<http://www.finalclap.com/faq/472-https-localhost-apache>
https://wiki.mozilla.org/Security/Server_Side_TLS
<https://wiki.evolix.org/HowtoSSL>